Safe Edges

AUDIT REPORT

2025

safeedges.in

STORK Audit Report By Safe Edges

Reviewed by: SafeEdges
Prepared for: Stork
Review Dates: 26/06/2025 - 04/07/2025

Protocol Summary

An oracle protocol that enables ultra-low latency connections between data providers and both on- and off-chain applications.

Identifier	Title	Severity	Details	Status
H-01	Tokens sent to the contract are not checked, allowing worthless tokens to pay for updates	Severity High	View Details	Resolved
M-01	Fees are never refunded if unspent	Severity Medium	View Details	Acknowledge
L-01	No way to update or revoke contract ownership	Severity Low	View Details	Resolved
L-02	Incomplete handling of `match` statements	Severity Low	View Details	Resolved
L-03	Signature malleability and replay attacks exist, though not exploitable	Severity Low	View Details	Resolved
I-01	Front-running during initialization	Severity Informational	View Details	Acknowledge

Issue Details

H-01 Tokens Sent to the Contract Are Not Checked, Allowing the utilization of Worthless Tokens to Pay for Updates

Severity: High

Description:
Subscribers receive updates from aggregators, and fees are required for every price update. However, on the Fuel ecosystem, every token is a native asset, allowing subscribers to bypass fees by creating their own custom but worthless tokens and sending the required amount to the `update_temporal_numeric_values_v1()` function to avoid paying with the intended token.

Here is the relevant function:

```
#[storage(read, write), payable]
fn update_temporal_numeric_values_v1(update_data: Vec<TemporalNumericValueInput>)
{
    // ... code snippet to update the tvns

    let required_fee = get_total_fee(num_updates);

    // @audit the msg asset id isn't checked, allowing any msg asset to be
accepted
    if std::context::msg_amount() < required_fee {
        log(StorkError::InsufficientFee);
        revert(0);
    }
}
```

While `msg_amount()` is checked, `msg_asset_id()` is never verified, allowing any token to satisfy the fee requirement. In practice, this could allow users to pay with valueless tokens rather than the protocol's designated token, undermining the economic model of the system and potentially preventing the protocol from funding aggregators or capturing revenue.

Recommendation:
Strictly validate the accepted token's asset ID to ensure only the designated token is accepted for payment.

Below is an example of the corrected implementation:

```
#[storage(read, write)]
#[payable]
fn update_temporal_numeric_values_v1(update_data: Vec<TemporalNumericValueInput>)
{
    // ... code to update tvns

    let required_fee = get_total_fee(num_updates);

    // Expected asset ID (e.g., BASE_ASSET_ID or your token's asset ID)
    let expected_asset_id = BASE_ASSET_ID; // or set to your token's ID

    // Check the correct asset and sufficient amount
    if std::context::msg_asset_id() != expected_asset_id {
        log(StorkError::InvalidPaymentAsset);
        revert(0);
    }

    if std::context::msg_amount() < required_fee {
        log(StorkError::InsufficientFee);
        revert(0);
    }
}
```

This ensures that only the intended token can be used to pay fees, preserving protocol integrity and economic value.

Status

Resloved : Commit

M-01 Fees are never refunded if unspent

Severity: Medium
Description:
The `update_temporal_numeric_valuess_v1()` function is a core function of the stork contract, where on each call the caller specifies the feeds and forwards the required amount of fees needed for the given updates, however given the scenario where some of the feeds to be updated are already updated(where no fee is charged), the excess being sent on the contract call remains in the contract without no refund to the caller even though no updates where made.

You can see the required_fee is the product of the stork fee and the number of updates made in that context.

```
        if (num_updates == 0) {
             log(StorkError::NoFreshUpdate);
             revert(0);
         }

         let required_fee = get_total_fee(num_updates);
```

In a scenario where updates are already made, the user funds sent will be stuck in the contract with no refunds to the user, leading to unintended loss for the callers.

```
    #[storage(read, write), payable]
    fn update_temporal_numeric_values_v1(update_data:
Vec<TemporalNumericValueInput>) {
        let mut num_updates = 0;
        let mut i = 0;
        while i < update_data.len() {
            let x = update_data.get(i).unwrap();
            let verified = verify_stork_signature(
                _stork_public_key(),
                x.id,
                x.temporal_numeric_value
                    .timestamp_ns,
                x.temporal_numeric_value
                    .quantized_value,
                x.publisher_merkle_root,
                x.value_compute_alg_hash,
                x.r,
                x.s,
                x.v,
```

```
            );

// code snippet ...

        let required_fee = get_total_fee(num_updates);
        if (std::context::msg_amount() < required_fee) {
            log(StorkError::InsufficientFee);
            revert(0);
        } //no refund if fee is too high.
    }
```

Recommendation:
Refund unspent fees after processing the update.

Status

Acknowledge

Client Comment

Most pushers will be running Stork's open source chain pusher, which should mitigate this edge case. Stork would rather not introduce two way payment capabilities at this time. Stork reserves the capability to address this further in a future upgrade.

L-1 No Way to Update or Revoke Contract Ownership

Severity: Low

Description:
There is no way to update or revoke ownership, even though the contract implements the `src5` standard. In scenarios where ownership must be transferred or revoked (e.g., team changes, compromised keys, governance upgrades), this limitation prevents any update to the owner address, making recovery or delegation impossible.

Recommendation:
Implement a secure ownership transfer mechanism using a two-step process (propose and accept) to avoid accidental or malicious changes.

For example:

1. **Propose transfer:**

```
#[storage(read, write)]
fn propose_owner(new_owner: Address) {
    onlyOwner();
    state.pending_owner = new_owner;
}
```

2. **Accept transfer:**

```
#[storage(read, write)]
fn accept_ownership() {
    if std::context::msg_sender() != state.pending_owner {
        revert(0);
    }
    state.owner = state.pending_owner;
    state.pending_owner = ZERO_ADDRESS;
}
```

This approach ensures ownership can be updated safely without requiring a contract redeployment.

Status

Resloved : Commit

Client Comment

Two step ownership transfer has been implemented, however Stork would rather not introduce ownership revocation at this time. Stork reserves the capability to implement ownership revocation in a future upgrade.

L2 Incomplete Handling of `match` Statements

Severity: Low

Description:
In the `only_owner()` call, not all `match` cases are handled safely, which can lead to unintended access to functions that are meant to be owner-restricted.

For example, if ownership is revoked to signal permissionlessness by the protocol, *anyone* will be able to call functions protected by `only_owner()`.

Below is the relevant code snippet:

```
#[storage(read)]
fn only_owner() {
    match storage.owner.read() {
        standards::src5::State::Uninitialized => {},
        standards::src5::State::Initialized(owner) => {
            require(msg_sender().unwrap() == owner, "Only Owner");
        },
        standards::src5::State::Revoked => {} // no logic for revoked state
    }
}
```

In this logic, the `Revoked` state simply does nothing, effectively allowing any caller through.

Recommendation:
Ensure exhaustive and secure handling of all `match` cases.

For example, explicitly revert in the `Revoked` state to prevent unintended access:

```
 standards::src5::State::Uninitialized => {},
// .. snippet...
standards::src5::State::Revoked => {
    revert(0);
}
```

Status

Resloved : Commit

L3 Signature Malleability and Replay Risk

Severity: Low

Description:
When obtaining a signature via `r`, `s`, and `v`, the `try_get_rsv_signature_from_parts()` function only checks whether `v` is `27` or `28`. However, it does not protect against signature malleability.

In Ethereum, all signatures are inherently malleable because every `s` value has a symmetrically corresponding `s` on the other side of the `secp256k1` curve. Without restricting the allowed `s` range, signatures can be manipulated.

Additionally, without including a nonce in each signature, the same signature can be replayed multiple times. Although this is **not directly exploitable** in this contract, since updates only occur if the current timestamp exceeds the previous one, it is considered best practice to enforce non-malleable signatures and include a nonce whenever using Ethereum-signed messages.

Impact:
Low, as this issue is not directly exploitable in the current implementation.

Recommendation:

1. **Restrict `s` to the lower half of the curve (EIP-2 constraints):**
   Require `s` to be less than or equal to `secp256k1n / 2`.

2. **Introduce a nonce per signature** to prevent replay attacks.

Below is an example of enhanced validation logic:

```
fn try_get_rsv_signature_from_parts(r: b256, s: b256, v: u8) -> Option<Secp256k1>
{
    // Reject malleable signatures by requiring s in lower range
    require(
        s <=
b256::from(0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0),
        "InvalidSignature"
    );
```

```
    // EIP-2 still allows signature malleability for ecrecover().
    // Remove this possibility and make the signature unique.
    // Appendix F in the Ethereum Yellow Paper
(https://ethereum.github.io/yellowpaper/paper.pdf):
    // Valid s: 0 < s < secp256k1n / 2 + 1
    // Valid v: {27, 28}

    match v {
        27 => {
            // Clear the highest bit
            let mask = b256::max()
                & !
(b256::from(0x8000000000000000000000000000000000000000000000000000000000000000));
            let s_cleared = s & mask;
            Some(Secp256k1::from((r, s_cleared)))
        }
        28 => {
            // Set the highest bit
            let mask =
b256::from(0x8000000000000000000000000000000000000000000000000000000000000000);
            let s_set = s | mask;
            Some(Secp256k1::from((r, s_set)))
        }
        _ => None,
    }
}
```

Status

Resloved : Commit

Additional Recommendation:
Include a **nonce** in each signature and store it in contract state to ensure each signed message can only be used once.

I-01 Front-running initialization

Severity: Informational
Description:
The call to `initialize()` sets core states required by the protocol to function, the issue stems from the ability for the function to be initialized before the real owner, either leading to grieving where the owner has to redeploy the contract where gas is wasted, or the owner is unaware leading to more issues in the future.

```
   #[storage(read, write)]
    fn initialize(
        initial_owner: Identity,
        stork_public_key: EvmAddress,
        single_update_fee_in_wei: u64,
    ) {
```

```
        require(!storage.initialized.read(), "Already initialized");
        require(!storage.initializing.read(), "Already initializing");
        storage.initializing.write(true);

        storage
            .owner
            .write(standards::src5::State::Initialized(initial_owner));

        set_single_update_fee_in_wei(single_update_fee_in_wei);
        set_stork_public_key(stork_public_key);
        storage.initialized.write(true);
    }
```

Recommendation:
Set a constant identity that is the only one permitted to call this function after deployment, or ensure a script is called to initialize it after contract creation

Status

Acknowledge

About SAFE EDGES

SafeEdges is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With six years of expertise, we've secured over 1000 projects globally, averting over $30 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

500+
Audits Completed

$3B
Secured

600k+
Lines of Code Audited